



ЗВІТ

Лабораторна робота № 5.2

Дисципліна: Базы даних та інформаційні системи

Тема: Практичне використання Aggregation Framework у MongoDB

Виконав

Студент 3 курсу

Групи МІТ-31

Факультету інформаційних технологій

Бабяк Володимир

Мета: Закріпити знання про основні стадії Aggregation Framework.

Навчитися будувати ефективні агрегаційні запити. Освоїти методи фільтрації, групування, сортування та обробки масивів у MongoDB. Практично працювати з \$match, \$group, \$sort, \$unwind, \$lookup, \$project. Аналізувати продуктивність агрегацій та оптимізувати запити.

Хід роботи

Варіант 3: Доставка їжі

Частина 1: Базові агрегаційні операції

1. Відфільтруйте замовлення за останні 3 місяці Використовуємо стадію \$match. Оскільки в минулій роботі ми створювали замовлення за травень 2024 року, встановимо дату так, щоб захопити ці дані:

```
< switched to db lab5
> db.orders.aggregate([
  {
    $match: {
      date: { $gte: new Date("2024-03-01") }
    }
  }
])
< {
  _id: ObjectId('6a05c898c022ebba7bdb0473'),
  orderNo: 'ORD001',
  clientPhone: '+380671234567',
  date: 2024-05-01T21:00:00.000Z,
  items: [
    {
      dish: 'Піца Маргарита',
      qty: 2,
      price: 250
    },
    {
      dish: 'Салат Цезар',
      qty: 2,
      price: 200
    }
  ],
  status: 'Pending'
}
{
  _id: ObjectId('6a05c898c022ebba7bdb0474'),
  orderNo: 'ORD002',
  clientPhone: '+380639998877',
  date: 2024-05-02T21:00:00.000Z,
  items: [
    {
      dish: 'Піца Маргарита',
      qty: 3,
      price: 250
    },
    {
      dish: 'Салат Цезар',
      qty: 1,
      price: 200
    }
  ],
  status: 'Cancelled'
}
{
  _id: ObjectId('6a05c898c022ebba7bdb0475'),
  orderNo: 'ORD003',
  clientPhone: '+380991112233',
  date: 2024-05-03T21:00:00.000Z,
```

2. Групування замовлень за місяцем Використовуємо стадію \$group та оператор \$month, щоб витягти місяць із дати, а потім рахуємо кількість замовлень у цьому місяці за допомогою \$sum:

```
Type "it" for more
> db.orders.aggregate([
  {
    $group: {
      _id: { $month: "$date" },
      totalOrders: { $sum: 1 }
    }
  }
])
< {
  _id: 4,
  totalOrders: 1
}
{
  _id: 5,
  totalOrders: 29
}
```

3. Сортування за сумою замовлення Оскільки в нашій структурі замовлення немає готового поля із загальною сумою (є лише масив items з кількістю та ціною кожної страви), ми спочатку обчислимо загальну суму для кожного замовлення за допомогою \$addFields, а потім відсортуємо результат за спаданням (\$sort: -1):

```

> db.orders.aggregate([
  {
    $addFields: {
      orderTotal: {
        $sum: {
          $map: {
            input: "$items",
            as: "item",
            in: { $multiply: ["$$item.qty", "$$item.price"] }
          }
        }
      }
    }
  },
  {
    $sort: { orderTotal: -1 }
  }
])

```

```

< {
  _id: ObjectId('6a05c898c022ebba7bdb0477'),
  orderNo: 'ORD005',
  clientPhone: '+380671234567',
  date: 2024-05-05T21:00:00.000Z,
  items: [
    {
      dish: 'Піца Маргарита',
      qty: 3,
      price: 250
    },
    {
      dish: 'Салат Цезар',
      qty: 2,
      price: 200
    }
  ],
  status: 'Cancelled',
  orderTotal: 1150
}
{
  _id: ObjectId('6a05c898c022ebba7bdb047d'),
  orderNo: 'ORD011',
  clientPhone: '+380991112233',
  date: 2024-05-11T21:00:00.000Z,
  items: [
    {
      dish: 'Піца Маргарита',
      qty: 3,
      price: 250
    },
    {
      dish: 'Салат Цезар',

```

Частина 2: Робота з масивами

Завдання 4. Розгорніть масив `items` у замовленнях Оператор `$unwind` "розпаковує" масив. Якщо в одному замовленні було 2 страви, цей запит розділить його на 2 окремих документи, в кожному з яких буде лише одна стравка з масиву. Це необхідно для подальшого аналізу кожної проданої позиції окремо.

```
> db.orders.aggregate([
  {
    $unwind: "$items"
  }
])
< {
  _id: ObjectId('6a05c898c022ebba7bdb0473'),
  orderNo: 'ORD001',
  clientPhone: '+380671234567',
  date: 2024-05-01T21:00:00.000Z,
  items: {
    dish: 'Піца Маргарита',
    qty: 2,
    price: 250
  },
  status: 'Pending'
}
{
  _id: ObjectId('6a05c898c022ebba7bdb0473'),
  orderNo: 'ORD001',
  clientPhone: '+380671234567',
  date: 2024-05-01T21:00:00.000Z,
  items: {
    dish: 'Салат Цезар',
    qty: 2,
    price: 200
  },
  status: 'Pending'
}
{
  _id: ObjectId('6a05c898c022ebba7bdb0474'),
  orderNo: 'ORD002',
  clientPhone: '+380639998877',
  date: 2024-05-02T21:00:00.000Z,
  items: {
    dish: 'Піца Маргарита',
    qty: 3,
    price: 250
  },
  status: 'Cancelled'
}
{
  _id: ObjectId('6a05c898c022ebba7bdb0474'),
  orderNo: 'ORD002',
  clientPhone: '+380639998877',
  date: 2024-05-02T21:00:00.000Z,
  items: {
    dish: 'Салат Цезар',
    qty: 1,
    price: 200
  },
  status: 'Cancelled'
}
```

Завдання 5. Підрахуйте кількість проданих одиниць товарів Тут ми будуємо конвеєр (pipeline) з трьох стадій: спочатку розгортаємо масив страв (\$unwind), потім групуємо їх за назвою страви (\$group), і за допомогою \$sum додаємо значення поля qty (кількість), щоб дізнатися, скільки всього порцій кожної страви було продано. Для краси звіту я додав стадію \$sort, щоб найпопулярніші страви були зверху.

```
> db.orders.aggregate([
  {
    $unwind: "$items"
  },
  {
    $group: {
      _id: "$items.dish",
      totalSoldUnits: { $sum: "$items.qty" }
    }
  },
  {
    $sort: { totalSoldUnits: -1 }
  }
])
< {
  _id: 'Піца Маргарита',
  totalSoldUnits: 60
}
{
  _id: 'Салат Цезар',
  totalSoldUnits: 45
}
```

Частина 3: З'єднання колекцій (\$lookup)

Завдання 6. Отримання інформації про клієнтів у замовленнях Цей запит бере кожне замовлення з колекції orders, шукає в колекції clients документ, де поле phone збігається з clientPhone із замовлення, і додає цю інформацію у новий масив clientDetails.

```
> db.orders.aggregate([
  {
    $lookup: {
      from: "clients",           // колекція, з якою з'єднуємо
      localField: "clientPhone", // поле в колекції orders
      foreignField: "phone",     // поле в колекції clients
      as: "clientDetails"       // назва нового поля для результату
    }
  }
])
< {
  _id: ObjectId('6a05c898c022ebbba7bdb0473'),
  orderNo: 'ORD001',
  clientPhone: '+380671234567',
  date: 2024-05-01T21:00:00.000Z,
  items: [
    {
      dish: 'Піца Маргарита',
      qty: 2,
      price: 250
    },
    {
      dish: 'Салат Цезар',
      qty: 2,
      price: 200
    }
  ],
  status: 'Pending',
  clientDetails: [
    {
      _id: ObjectId('6a05c7d1c022ebbba7bdb045b'),
      name: 'Марія Петренко',
      phone: '+380671234567',
      district: 'Печерський'
    }
  ]
}
{
  _id: ObjectId('6a05c898c022ebbba7bdb0474'),
  orderNo: 'ORD002',
  clientPhone: '+380639998877',
  date: 2024-05-02T21:00:00.000Z,
  items: [
    {
      dish: 'Піца Маргарита',
      qty: 3,
      price: 250
    },
    {
      dish: 'Салат Цезар',
      qty: 1,
      price: 200
    }
  ]
}
```


Завдання 7. Визначте найбільш активних клієнтів Тут ми будемо цікавіший конвеєр: спочатку групуємо всі замовлення за номером телефону клієнта (\$group) і рахуємо їхню кількість. Потім сортуємо клієнтів від найактивніших до найменш активних (\$sort). А наприкінці додаємо \$lookup, щоб побачити не просто номер телефону лідера, а його повне ім'я та дані з колекції clients.

```
Type "it" for more
> db.orders.aggregate([
  {
    $group: {
      _id: "$clientPhone",
      totalOrders: { $sum: 1 }
    }
  },
  {
    $sort: { totalOrders: -1 }
  },
  {
    $lookup: {
      from: "clients",
      localField: "_id",
      foreignField: "phone",
      as: "clientInfo"
    }
  }
])
< {
  _id: '+380639998877',
  totalOrders: 8,
  clientInfo: [
    {
      _id: ObjectId('6a05c7d1c022ebba7bdb045c'),
      name: 'Дмитро Коваленко',
      phone: '+380639998877',
      district: 'Оболонський'
    }
  ]
}
{
  _id: '+380671234567',
  totalOrders: 8,
  clientInfo: [
    {
      _id: ObjectId('6a05c7d1c022ebba7bdb045b'),
      name: 'Марія Петренко',
      phone: '+380671234567',
      district: 'Печерський'
    }
  ]
}
/
```

Частина 4: Оптимізація запитів

Щоб перевірити продуктивність агрегаційного запиту (скільки часу він виконується і скільки документів переглядає), до нього додають метод `.explain("executionStats")`.

Завдання 8. Перевірте продуктивність запиту Перевіримо, як база даних шукає виконані замовлення та групує їх за клієнтами (без використання спеціального індексу на цій стадії, якщо ми його ще не робили).

```

> db.orders.explain("executionStats").aggregate([
  { $match: { status: "Completed" } },
  { $group: { _id: "$clientPhone", totalOrders: { $sum: 1 } } }
])
< {
  explainVersion: '1',
  stages: [
    {
      '$cursor': {
        queryPlanner: {
          namespace: 'lab5.orders',
          indexFilterSet: false,
          parsedQuery: {
            status: {
              '$eq': 'Completed'
            }
          },
          queryHash: 'ED87DEDE',
          planCacheKey: 'ED87DEDE',
          optimizationTimeMillis: 0,
          maxIndexedOrSolutionsReached: false,
          maxIndexedAndSolutionsReached: false,
          maxScansToExplodeReached: false,
          winningPlan: {
            stage: 'PROJECTION_SIMPLE',
            transformBy: {
              clientPhone: 1,
              _id: 0
            },
            inputStage: {
              stage: 'COLLSCAN',
              filter: {
                status: {
                  '$eq': 'Completed'
                }
              },
              direction: 'forward'
            }
          },
          rejectedPlans: []
        },
        executionStats: {
          executionSuccess: true,
          nReturned: 10,
          executionTimeMillis: 0,
          totalKeysExamined: 0,
          totalDocsExamined: 30,
          executionStages: {
            stage: 'PROJECTION_SIMPLE',
            nReturned: 10,
            executionTimeMillisEstimate: 0,

```

Завдання 9. Оптимізуйте агрегаційний запит Найкращий спосіб оптимізувати агрегацію — додати індекс на ті поля, які використовуються у стадії \$match на самому початку конвеєра. Ми створимо індекс для поля status, а потім знову запустимо той самий запит, щоб показати оптимізацію:

```

> db.orders.createIndex({ status: 1 });

db.orders.explain("executionStats").aggregate([
  { $match: { status: "Completed" } },
  { $group: { _id: "$clientPhone", totalOrders: { $sum: 1 } } }
])
< {
  explainVersion: '1',
  stages: [
    {
      '$cursor': {
        queryPlanner: {
          namespace: 'lab5.orders',
          indexFilterSet: false,
          parsedQuery: {
            status: {
              '$eq': 'Completed'
            }
          },
          queryHash: 'ED87DEDE',
          planCacheKey: 'C6EB6B5D',
          optimizationTimeMillis: 0,
          maxIndexedOrSolutionsReached: false,
          maxIndexedAndSolutionsReached: false,
          maxScansToExplodeReached: false,
          winningPlan: {
            stage: 'PROJECTION_SIMPLE',
            transformBy: {
              clientPhone: 1,
              _id: 0
            },
          },
          inputStage: {
            stage: 'FETCH',
            inputStage: {
              stage: 'IXSCAN',
              keyPattern: {
                status: 1
              },
              indexName: 'status_1',
              isMultiKey: false,
              multiKeyPaths: {
                status: []
              },
              isUnique: false,
              isSparse: false,
              isPartial: false,
              indexVersion: 2,
              direction: 'forward',
              indexBounds: {
                status: [
                  '["Completed", "Completed"]'
                ]
              }
            }
          }
        }
      }
    }
  ]
}

```

Додаткові завдання

Завдання 10. Визначте категорії товарів із найбільшою кількістю продажів Оскільки в нас категорія — це "кухня" (cuisine), яка знаходиться в колекції dishes, а кількість продажів — у колекції orders, нам потрібно їх об'єднати! Це дуже крутий комбінований запит: розгортаємо масив, підтягуємо дані про страву, групуємо за кухнею і сортуємо.

```
> db.orders.aggregate([
  { $unwind: "$items" },
  { $lookup: { from: "dishes", localField: "items.dish", foreignField: "name", as: "dishInfo" } },
  { $unwind: "$dishInfo" },
  { $group: { _id: "$dishInfo.cuisine", totalSold: { $sum: "$items.qty" } } },
  { $sort: { totalSold: -1 } }
])
< {
  _id: 'Італійська',
  totalSold: 68
}
{
  _id: 'Європейська',
  totalSold: 45
}
lab5>
```

Завдання 11. Розрахуйте середню ціну товарів у кожній категорії Для цього достатньо колекції страв. Групуємо страви за кухнею і використовуємо математичний оператор \$avg для розрахунку середньої ціни:

```

> db.dishes.aggregate([
  { $group: { _id: "$cuisine", avgPrice: { $avg: "$price" } } }
])
< {
  _id: 'Японська',
  avgPrice: 335
}
{
  _id: 'Азіатська',
  avgPrice: 345
}
{
  _id: 'Італійська',
  avgPrice: 273.3333333333333
}
{
  _id: 'Американська',
  avgPrice: 285
}
{
  _id: 'Європейська',
  avgPrice: 276.6666666666667
}
{
  _id: 'Українська',
  avgPrice: 148
}
}
lab5>

```

Завдання 12. Знайдіть користувачів, які зробили більше одного замовлення Тут ми спочатку рахуємо замовлення кожного клієнта (через \$group), а потім відфільтровуємо лише тих, у кого кількість більша за 1 (через \$match):

```
> db.orders.aggregate([
  { $group: { _id: "$clientPhone", orderCount: { $sum: 1 } } },
  { $match: { orderCount: { $gt: 1 } } }
])
< {
  _id: '+380639998877',
  orderCount: 8
}
{
  _id: '+380991112233',
  orderCount: 7
}
{
  _id: '+380671234567',
  orderCount: 8
}
{
  _id: '+380501112233',
  orderCount: 7
}
lab5>
```

Висновок

Під час виконання лабораторної роботи № 5.2 я на практиці розібрався з Aggregation Framework у MongoDB. Навчився будувати конвеєри запитів, використовуючи базові стадії \$match, \$group та \$sort.

Новим для мене стала робота з вкладеними масивами через \$unwind та об'єднання даних з різних колекцій за допомогою \$lookup (аналог JOIN у реляційних БД). Найбільше труднощів було з тим, щоб правильно вибудувати правильну послідовність стадій, оскільки від цього залежить фінальний результат групування.

Також перевінив швидкість виконання агрегацій через explain. Як показала практика, для оптимізації найефективніше ставити фільтрацію \$match на самому початку пайплайну та використовувати індекси — це суттєво зменшує кількість переглянутих документів і пришвидшує запит.